# Asset Pipeline Guide: From PNG, TMX and Text to Chip RAM

A companion guide to Building an AMIGA Game Engine

Richard Shannon

AmigaGameEngine v1.0

## Contents

**1. The Pipeline at a Glance** **1**

**2. Formats the Hardware Reads** **2**
Interleaved bitplanes . . . 2
Cookie-cut masks . . . 2
White flash sheets . . . 2
Palettes . . . 2
Sheet geometry . . . 2

**3. Levels: Tiled to Amiga** **3**
Layers, by name . . . 3
Map properties . . . 3
Outputs . . . 4

**4. Sprites, BOBs and Screens** **4**

**5. Sound and Music** **4**

**6. Data Tables** **4**

**7. Compressed Data** **5**

**8. Chip RAM or Fast RAM** **5**

**9. The Golden Tests** **5**

**10. Adding an Asset: Checklist** **5**

Every graphic, level, sound and lookup table in Duck Rescue starts as a file you can open in a normal tool: a PNG, a Tiled `.tmx` map, or a text file. Python converters turn those into the raw formats the Amiga hardware reads directly, and `main.asm` includes the results with `incbin`. Nothing is converted on the Amiga at load time, except one compressed screen and the mirrored player frames.

This guide covers the formats the hardware expects, each converter, where its output goes in memory, and how the golden tests catch an accidental change. The converters live in `games/duck_rescue/tools/`; a new game copies that folder (see the Making a Game Guide).

## 1. The Pipeline at a Glance

`build.py` runs the converters in the order of its `ASSET_PIPELINE` list, from the game's own `tools/` folder, before assembling:

| Order | Script | Runs | Produces |
|---|---|---|---|
| 1 | `convert_assets.py` | `--assets` only | title logo, Copper sky gradient (manifest-driven) |

| Order | Script | Runs | Produces |
|---|---|---|---|
| 2 | `convert_duck_banner.py` | `--assets` only | title banner, game-complete duckling |
| 3 | `generate_sfx.py` | `--assets` only | sound effects (synthesised) |
| 4 | `convert_data_tables.py` | every build | `sin.bin`, `quadratic.bin` from `.txt` |
| 5 | `export_level.py` | every build | tile sheet, layer maps, logic maps, `level_XX_entities.asm` |
| 6 | `convert_player_bob.py` | every build | Mia's BOB sheet, mask, white flash, palette |
| 7 | `convert_animals.py` | every build | animal friend sheet and mask |
| 8 | `convert_enemies.py` | every build | enemy sheets, white flash, dizzy stars |
| 9 | `convert_effects.py` | every build | landing dust puff |
| 10 | `convert_headshots.py` | every build | level-complete portrait (5 planes) |

`--assets` steps are slow or rarely change, so they run only when you ask:

```
python build.py                 # convert, assemble, link uae/dh0/main
python build.py --assets        # also regenerate the --assets-only outputs
```

A script that is missing from a game's `tools/` folder is skipped. One converter runs by hand: `convert_loading_screen.py` (5-plane loading screen, compressed with ZX0).

# 2. Formats the Hardware Reads

## Interleaved bitplanes

Every BOB, tile sheet and screen uses **interleaved** bitplanes: each line of the image holds plane 0's bytes, then plane 1's, and so on. A 64-pixel-wide, 4-plane sheet has lines of 4 × 8 = 32 bytes. One Blitter pass then covers all planes by treating each line as four, which is why every engine blit height is "lines × planes".

The shared packer is `pack_row_planes()` in `amiga_planar.py`: it takes one row of palette indices and returns the interleaved graphic and its mask. Every converter uses it, so the layout is defined in one place.

## Cookie-cut masks

A mask has the **same size and layout as the graphic**. A pixel is solid when its palette index is not 0, and that bit is repeated in every plane's slice of the line. The Blitter reads mask (A), graphic (B) and background (C) as one interleaved stream with the minterm `$CA` (D = AB + ¬AC), so the mask has to line up plane for plane with the graphic.

## White flash sheets

`_white.raw` files are the same shapes with every solid pixel set to one colour (index 3, white). The engine blits them instead of the normal sheet for hit flashes and invulnerability.

## Palettes

A `.pal` file is big-endian `$0RGB` words, one per colour: 16 words (32 bytes) for a 4-plane image, 32 words (64 bytes) for 5 planes. The game's 16 playfield colours are defined once, in `game_palette.json`, as 4-bit RGB per channel, and every converter loads them from there. Colour 0 is transparent in BOBs and sheets.

## Sheet geometry

| Asset | Cell | Sheet | Bytes (raw = mask) |
|---|---|---|---|
| Player (Mia) | 32 × 24 (24 pixels used) | 128 × 144, 6 actions × 4 frames | 9,216 |
| Enemies | 16 × 16 | 64 × *h*, one type per row of 4 frames | 512 per type |
| Friends | 16 × 16 | 64 × 48, 3 animals × 4 frames | 1,536 |
| Dizzy stars | 16 × 16 | 64 × 16, 4 frames | 512 |
| Landing puff | 16 × 16 | 64 × 32, 7 frames (8th cell empty) | 1,024 |
| Tiles | 16 × 16 | 176 × 80, 11 × 5 tiles | 7,040 (+ 32-byte palette on the raw) |

Every engine sprite sheet has **four frames to a row of 64 pixels**: `entities.asm` computes frame addresses from that.

Only right-facing player frames are stored. `PlayerInitFlippedSprites` (`bob_mirror.asm`) mirrors them at boot into Chip RAM BSS buffers (`PlayerLeftRaw`, `PlayerLeftMsk`, `PlayerLeftWhiteRaw`). This halves the stored sheet at the cost of a one-off conversion at startup.

# 3. Levels: Tiled to Amiga

`export_level.py` reads each `assets/levels/Level_XX/Level_XX.tmx` and its tilesets.

## Layers, by name

The exporter decides each layer's role from its name, so name layers clearly:

| Role | Name contains | Becomes |
|---|---|---|
| Background | `background`, `scenery`, `bg`, `back`, `decor` | `-background.map` |
| Platforms | `platform`, `solid`, `floor`, `ground` | `-platform.map` and solid cells in the logic map |
| Ladders | `ladder`, `climb`, `vine` | `-ladder.map` and ladder cells |
| Foreground | `foreground`, `fg` | `-foreground.map`; stamped in front of the player |
| Water | `water` | `-water.map`; the starting waterline |
| Entities | `entit`, `actor`, `spawn`, `object`, `item` | the player start, crates, enemies, friends and pickups in `level_XX_entities.asm` |

## Map properties

| Property | Default | Meaning |
|---|---|---|
| `title`, `sub_title` | from the file name | level banner text |
| `music` | `LevelMod` | module to play |
| `palette`, `tileset_raw`, `tileset_msk` | level prefix, `GameTilesRaw/Msk` | data symbols the `LevelDef` points at |
| `initial_camera_y` / `initial_row` | centred on the player | start camera |
| `min_camera_y`, `max_camera_y` | 0, rows × 16 − 216 | camera limits |
| `cam_margin_top`, `cam_margin_bottom` | 16, 176 | camera dead zone |
| `password` | `ACORN` + level number | level access code |

Enemy objects take `type` (1–8), `speed` and `patrol_min_x`/`patrol_max_x` (default: 64 pixels either side of the object). The player start takes `dir` (`left` or `right`). Tiles in the tilesets carry collision through a `type` (`solid`, `ladder`, `hazard`, `platform` for one-way) or boolean `solid`/`ladder` properties.

### Outputs

| File | Format | Used for |
|---|---|---|
| `tiles/tileset-compact_176x80.raw` / `.msk` | 4-plane interleaved, palette appended | every tile blit |
| `Level_XX-<layer>.map` | 8-byte header (width, height as big-endian longs), then one big-endian word per tile | drawing the rows (`TilemapRenderOneRowAllLayers`) |
| `Level_XX.map` | the same format, layers merged | composite view |
| `Level_XX-gamemap.bin` | one byte per cell (`BLOCK_*`) | collision and logic (`GameMap`) |
| `include/levels/level_XX_entities.asm` | generated assembly | the `LevelDef`: dimensions, camera, palette, layer pointers, entity lists |

The exporter rejects a map shorter than 13 rows (it would not fill the display) or taller than 50 rows (the Chip RAM budget, `MAX_SCREEN_ROWS`). The width is always 20 tiles.

## 4. Sprites, BOBs and Screens

| Script | Source | Notes |
|---|---|---|
| `convert_player_bob.py` | `player_master_female_128x144.png` | rows: idle, walk, attack, death, ladder, fall; `--preview` writes a contact sheet |
| `convert_enemies.py` | `enemies_16x16.png`, `dizzy_stars_64x16.png` | writes the full sheet and the sheet the game uses (`enemies_64x48`) |
| `convert_animals.py` | `animals_16x16.png` | 3 animals × 4 frames |
| `convert_effects.py` | `smoke_64x32.png` | every opaque pixel must already be a palette colour |
| `convert_headshots.py` | 128 × 64 PNG, two 64 × 64 frames | 5 planes, 32-colour palette, COLOR31 white |
| `convert_duck_banner.py` | 280 × 96 RGBA banner | splits title text and duckling |
| `convert_assets.py` | manifest at the bottom of the file | `planar` (any plane count, colours ranked by frequency), `1plane`, `star`, `copper_sky` |

To add a simple image, add an entry to the `ASSETS` manifest in `convert_assets.py`; most cases need no new code.

## 5. Sound and Music

**Sound effects** come from `generate_sfx.py`, which synthesises every effect: there are no source samples. The output is 8-bit signed PCM in `assets/fx/*.raw` for 11,025 Hz playback (Paula period 322 on PAL). Each file starts with a zero word, which PTPlayer needs as an idle loop, and has an even length. WAV previews go to `scratch/sfx_preview/` for listening on a PC.

**Music** is ProTracker `.mod` files, included as they are. Paula plays samples by DMA, so in-game modules live in Chip RAM. The title module is held in Fast RAM and copied into spare Chip RAM (inside `NonDisplayScreen`) by `TitleSetup`, because the title screen does not use the level buffers.

## 6. Data Tables

`convert_data_tables.py` turns every `assets/data/*.txt` into a `.bin` of big-endian words on every build.

- `;` starts a comment.
- Values are decimal, or hex with `$`, separated by spaces or commas.
- Each value must fit in a word (−32768..65535).
- The line `; entries: N` is **required**, and the build fails if the count differs. The engine derives sizes from table lengths (`SINE_ANGLES = (SinusEnd-Sinus)/2`), so a short table would quietly change behaviour.

A new table needs only a new `.txt` file and an `incbin` between two labels in `main.asm`.

## 7. Compressed Data

The loading screen (320 × 200, 5 planes, 40,000 bytes) is stored ZX0-compressed (`loadingscreen.zx0`) and unpacked into Chip RAM by `zx0_faster.asm`. ZX0 suits data that is unpacked once and is large compared to the unpacker. Graphics that are blitted every frame stay uncompressed in Chip RAM.

## 8. Chip RAM or Fast RAM

The custom chips can only read Chip RAM. Put data in the section that matches who reads it:

| Reader | Section | Examples |
|---|---|---|
| Blitter, bitplane, sprite, Copper or audio DMA | `data_chip,data_c` / `mem_chip,bss_c` | tile sheets, BOB sheets and masks, Copper lists, HUD sprites, in-game modules, sound effects, screen buffers |
| CPU only | `data_fast,data` / `mem_fast,bss` | palettes (copied into Copper lists), maps, `LevelDefs`, sine and easing tables, compressed data, the title module before staging |

Sections without the `_c` suffix may be loaded into any memory: Fast RAM, an A500's trapdoor slow RAM or, on a machine with neither, Chip RAM. So the split costs nothing on a bare machine and frees Chip RAM on an expanded one. The current totals and headroom are in the README and `chip_ram_report.py`.

## 9. The Golden Tests

Every converter is deterministic: the same inputs always give byte-identical outputs. `tests/test_converters_golden.py` runs the whole pipeline in a scratch copy of the game and compares a SHA-256 of every output with `tests/golden/<game>_converters.json`, so an accidental change to a converter or an asset fails the tests.

When you change art or a converter on purpose:

```
python tests/update_golden.py        # re-record the converter outputs
git diff tests/golden                # check only the files you meant changed
```

Rules for converter code (see the Coding Conventions Guide):

- Seed any randomness and sort any sets, so output never varies.
- Always write the output file, even when its contents are unchanged, so the golden test sees it.
- Start every script with a docstring saying what it converts, from where to where, and how to run it. `tool_cli.run_tool()` turns it into the script's `--help` and turns common errors into one-line messages.
- Load the palette from `game_palette.json`; never hard-code colours.

## 10. Adding an Asset: Checklist

1. **Source:** a PNG in palette colours, a Tiled layer or object, or a `.txt` table under `assets/`.
2. **Converter:** an entry in the `convert_assets.py` manifest, an existing converter, or a new script added to `ASSET_PIPELINE` in `build.py`.
3. **Section:** Chip RAM if any DMA reads it, Fast RAM otherwise (§8).
4. **Include:** `incbin` it in `main.asm` between a start label and, for variable-size data, an end label.
5. **Goldens:** run `python tests/update_golden.py` and review the diff.
6. **Budget:** run `chip_ram_report.py` and check the Chip RAM headroom.